

Lecture 7

DATAPATHS

ALU DESIGN

Introduction

- In the previous lecture, the separation of a design of digital systems into a Datapath that implements microoperations and a control unit that determines the sequence of microoperations was introduced.
- In this lecture, we continue by defining a generic computer datapath that implements register transfer microoperations and serves as a framework for the design of detailed processing logic.
- The concept of a control word provides a tie between the datapath and the control unit associated with it.
- The generic datapath combined with a control unit and memory forms a programmable system— in this case, a simple computer.

DATAPATHS

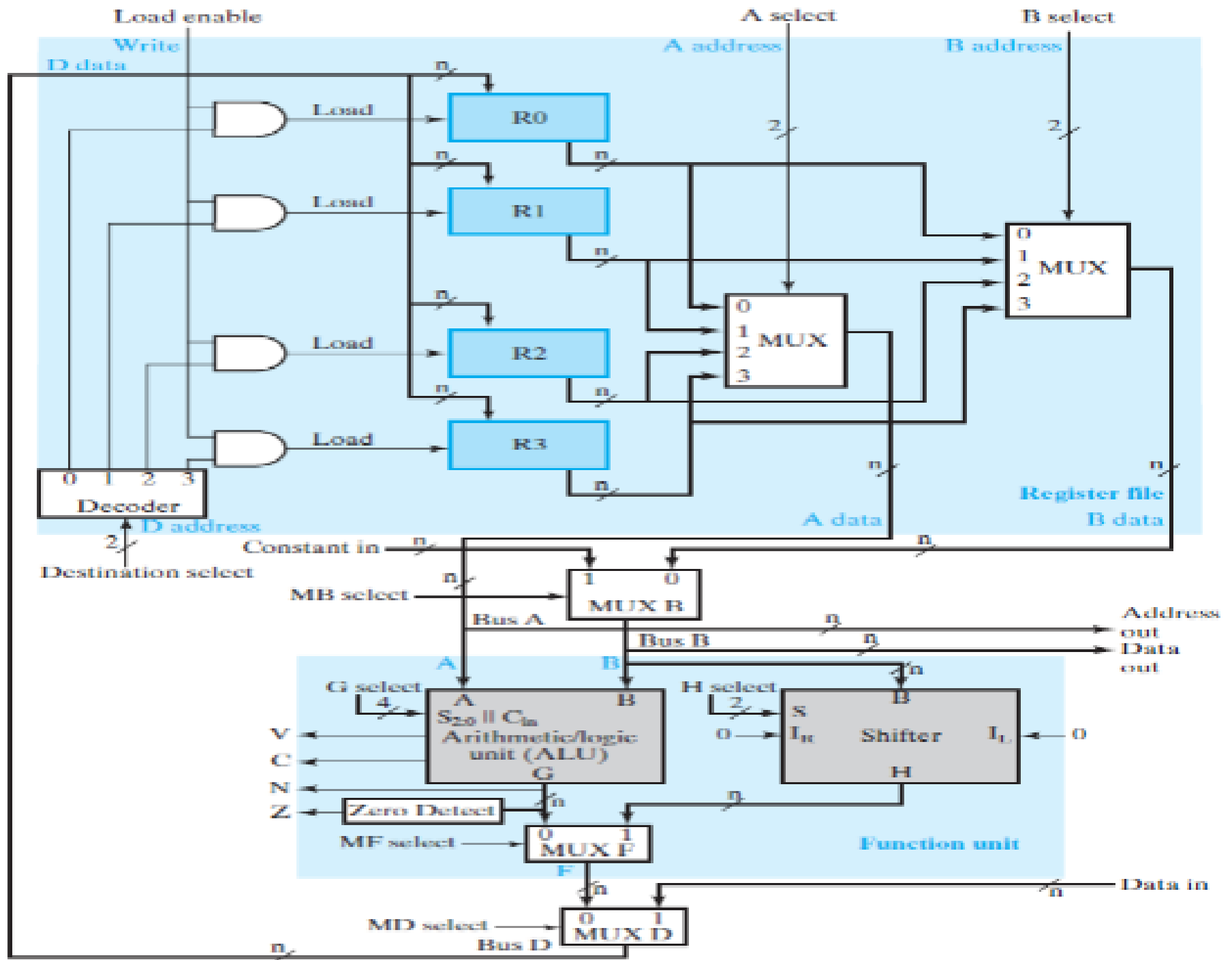
- Instead of having each individual register perform its microoperations directly, computer systems often employ a number of storage registers in conjunction with a shared operation unit called an *arithmetic/logic unit*, abbreviated ALU.
- To perform a microoperation, the contents of specified source registers are applied to the inputs of the shared ALU.
- The ALU performs an operation, and the result of this operation is transferred to a destination register.

Cont.,

- The datapath is defined by three basic components:
 1. A set of registers,
 2. The microoperations performed on data stored in the registers, and
 3. The control interface.
- The control unit provides signals that control the microoperations performed in the datapath and in other components of the system, such as memories.
- The rest of this chapter is concerned with the organization and design of computer datapaths and associated control units used to implement simple computers.

A simple bus-based datapath

- The specification for a computer consists of a description of its appearance to a programmer at the lowest level, its instruction set architecture (ISA).
- For instance, a simple bus-based datapath with four registers, an ALU, and a shifter is shown in the following Figure.



Cont.,

- Each register is connected to two multiplexers to form ALU and shifter input buses *A* and *B*.
- The select inputs on each multiplexer select one register for the corresponding bus.
- For *Bus B*, there is an additional multiplexer, MUX B, so that constants can be brought into the Datapath from outside using *Constant in*.
- *Bus B* also connects to Data out, to send data outside the Datapath to other components of the system, such as memory or input–output.

Cont.,

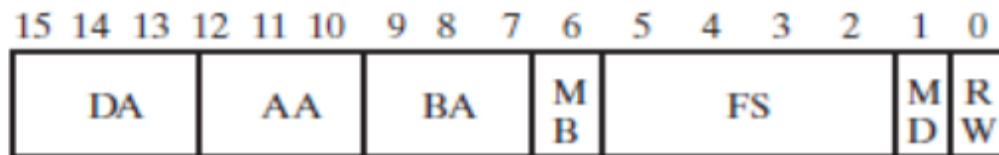
- Likewise, Bus *A* connects to Address out, to send address information outside of the datapath for memory or input–output.
- Arithmetic and logic microoperations are performed on the operands on the *A* and *B* buses by the ALU.
- The *G* select inputs select the microoperation to be performed by the ALU.
- The shift microoperations are performed on data on Bus *B* by the shifter.

Cont.,

- The *H* select input either passes the operand on *Bus B* directly through to the shifter output or selects a shift microoperation.
- MUX F selects the output of the ALU or the output of the shifter.
- MUX D selects the output of MUX F or external data on input *Data in* to be applied to *Bus D*.
- The latter is connected to the inputs of all the registers.
- The destination select inputs determine which register is loaded with the data on *Bus D*.

Cont.,

- Since the select inputs are decoded, only one register Load signal is active for any transfer of data into a register from *Bus D*.
- A Load enable signal can force all register Load signals to 0 using AND gates.
- The control unit for the datapath directs the information flow through the buses, the ALU, the shifter, and the registers by applying signals to the select inputs (control word).



(b) Control word

Example

- Specify the control word to perform the following microoperation:

$$R1 \leftarrow R2 + R3$$

Solution:

AA = 10, BA = 11, MB = 0,

G = ADD code, MF = 0, MD = 0,

Destination select = 01,

Load enable = 1.

DATAPATH REPRESENTATION

- A typical Datapath has more than four registers. Indeed, computers with 32 or more registers are common.
- The construction of a bus system with a large number of registers requires different techniques.
- A set of registers having common microoperations performed on them may be organized into a *register file*.
- The typical register file is a special type of fast memory that permits one or more words to be read and one or more words to be written, all simultaneously.

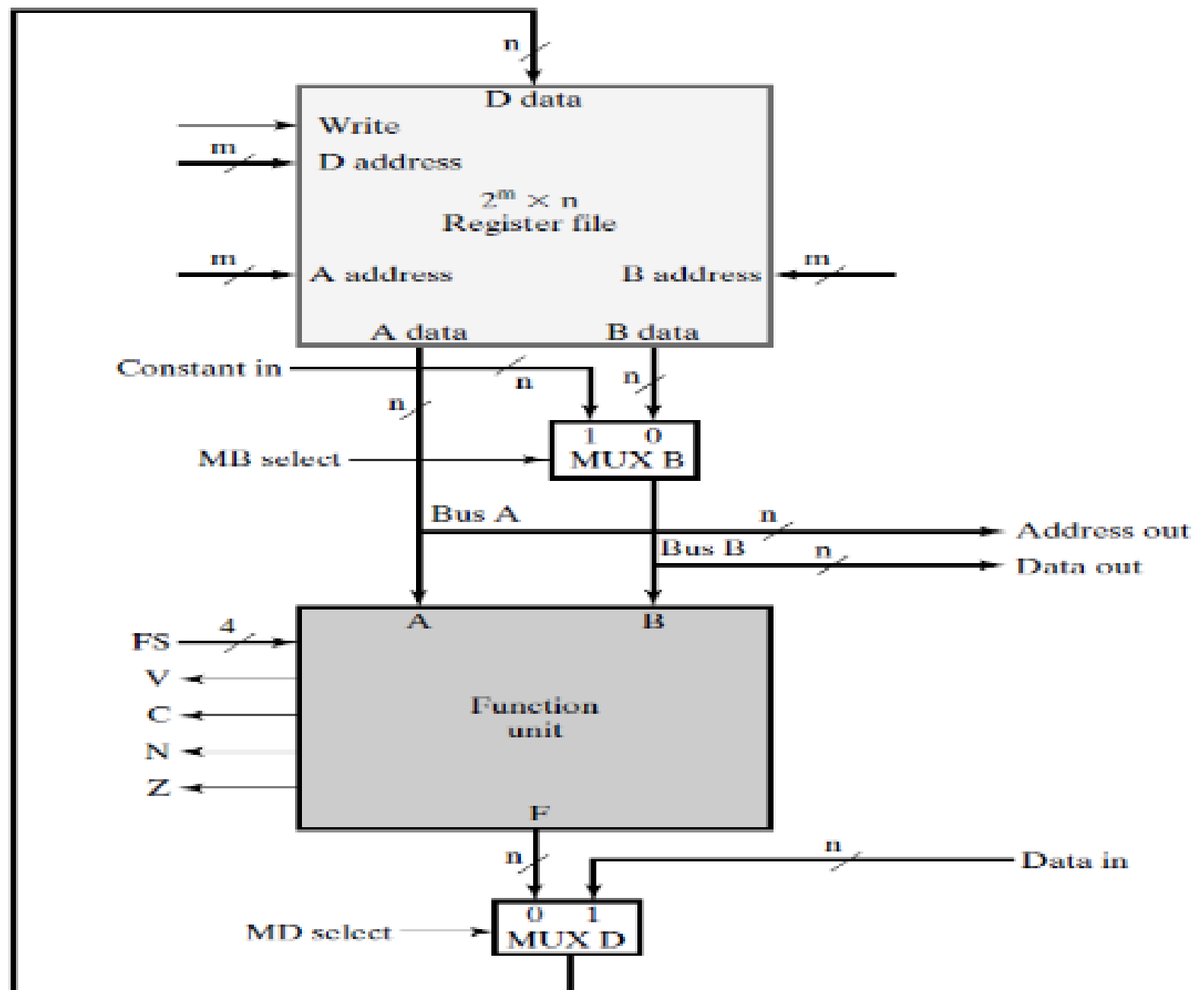
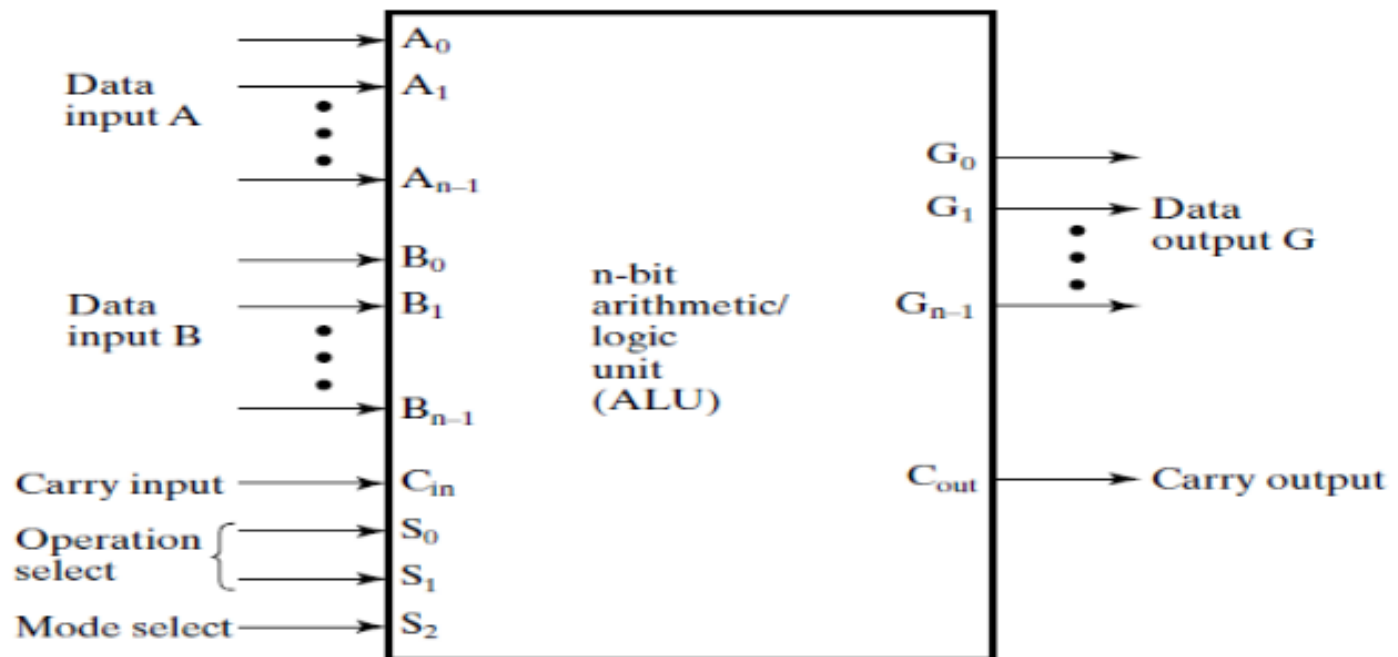


FIGURE 10
Block Diagram of Datapath Using the Register File and Function Unit

THE ARITHMETIC/LOGIC UNIT

- The ALU is a combinational circuit that performs a set of basic arithmetic and logic microoperations.



□ **FIGURE 2**
Symbol for an n -Bit ALU

Cont.,

- Where, mode select input S_2 distinguishes between arithmetic and logic operations.
- The n data inputs from A are combined with the n data inputs from B to generate the result of an operation at the G outputs.
- The two Operation select inputs S_1 and S_0 and the Carry input C_{in} specify the eight arithmetic operations with S_2 at 0.
- Operand select input S_0 and S_1 specify the four logic operations with S_2 at 1 and $C_{in} = 0$.

Design of ALU

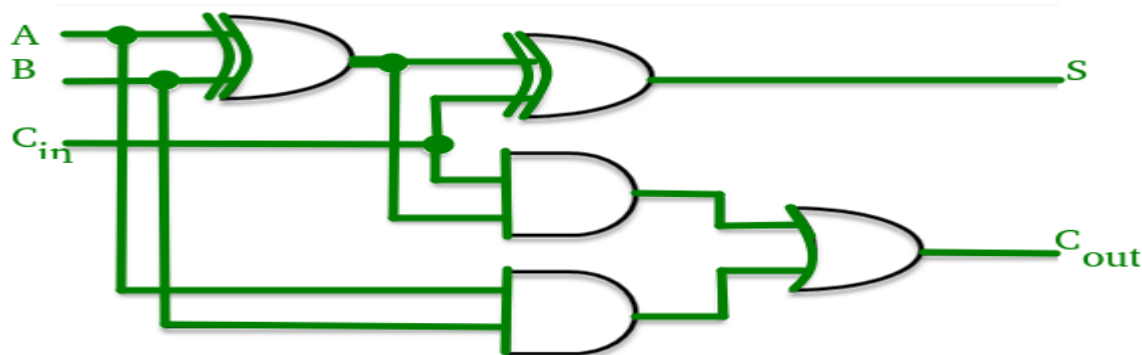
- The design steps of an ALU:
 - 1- Design the arithmetic section.
 - 2- Design the logic section,
 - 3- Combine the two sections to form the ALU.
- Where, the basic component of an ALU is the parallel adder.

Parallel Adder

- The basic architecture of the parallel adder is the Full Adder with carry lookahead circuits.
- Full Adder circuit is as shown:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B)).$$



Cont.,

- Consider the full adder circuit shown above with corresponding truth table. If we define two variables as carry generate G_i and carry propagate P_i then,

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

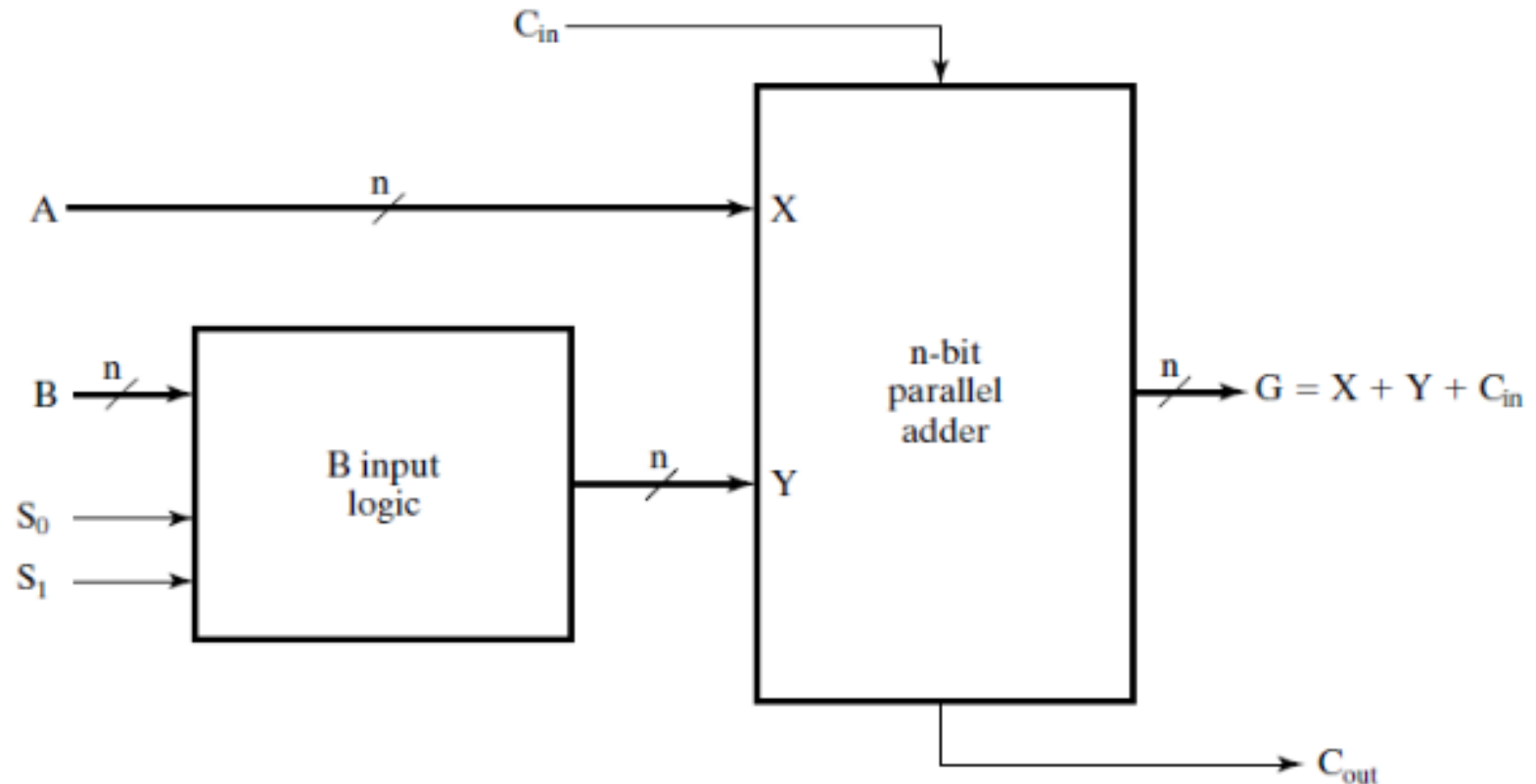
- The sum output and carry output can be expressed as

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

- Where G_i is a carry generate which produces the carry when both A_i, B_i are one regardless of the input carry.
- P_i is a carry propagate and it is associate with the propagation of carry from C_i to C_{i+1} .

Arithmetic Circuit



□ **FIGURE 3**
Block Diagram of an Arithmetic Circuit

- $1111 = -1D$
- $0000 + 1 = 0001 = 1D$
- 0001
- 1111

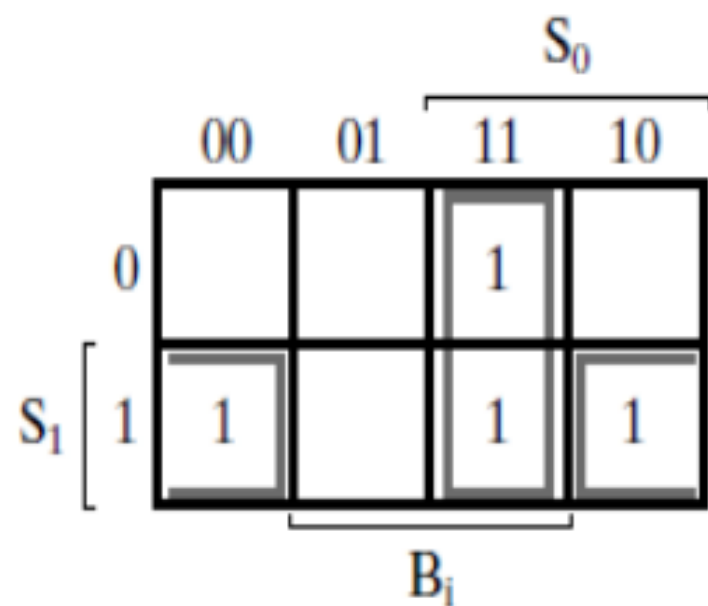
□ **TABLE 1**
Function Table for Arithmetic Circuit

Select		Input	$G = (A \mid Y \mid C_{in})$	
S_1	S_0	Y	$C_{in} = 0$	$C_{in} = 1$
0	0	all 0s	$G = A$ (transfer)	$G = A + 1$ (increment)
0	1	B	$G = A + B$ (add)	$G = A + B + 1$
1	0	$\bar{B}$	$G = A + \bar{B}$	$G = A + \bar{B} + 1$ (subtract)
1	1	all 1s	$G = A - 1$ (decrement)	$G = A$ (transfer)

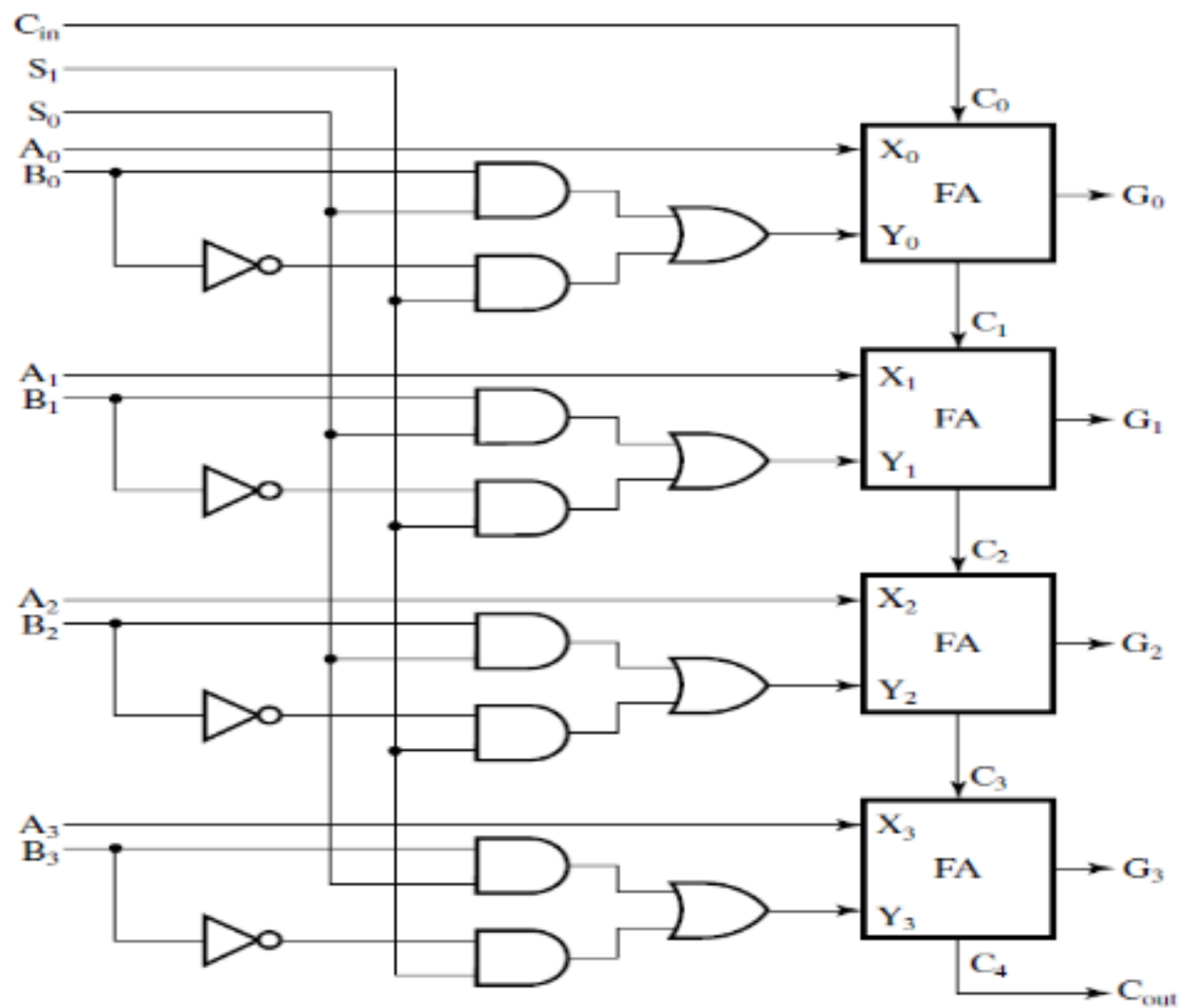
Where, $X = A$, and $G = X + Y + C_{in}$

Inputs			Output
S_1	S_0	B_i	Y_i
0	0	0	0 $Y_i = 0$
0	0	1	0
0	1	0	0 $Y_i = B_i$
0	1	1	1
1	0	0	1 $Y_i = \overline{B_i}$
1	0	1	0
1	1	0	1 $Y_i = 1$
1	1	1	1

(a) Truth table



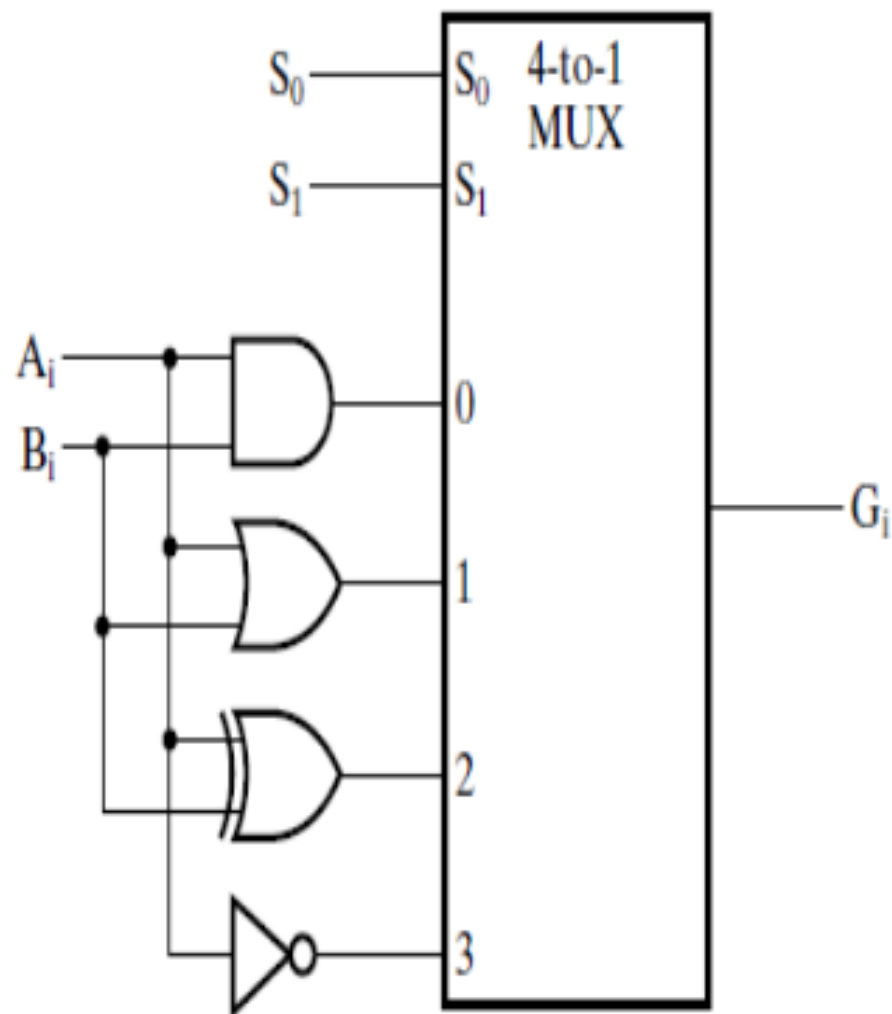
(b) Map simplification:
 $Y_i = B_i S_0 + \overline{B_i} S_1$



Logic Diagram of a 4-Bit Arithmetic Circuit

Logic Circuit

- The logic microoperations manipulate the bits of the operands by treating each bit in a register as a binary variable, giving bitwise operations.
- There are four commonly used logic operations—AND, OR, XOR (exclusive-OR), and NOT—from which others can be conveniently derived.
- For logic operations, C_{in} is inhibited to 0, by the following equation: $C_{in} = S_2^- \cdot C_i$
- Therefore, the logic operations are half of the arithmetic operations

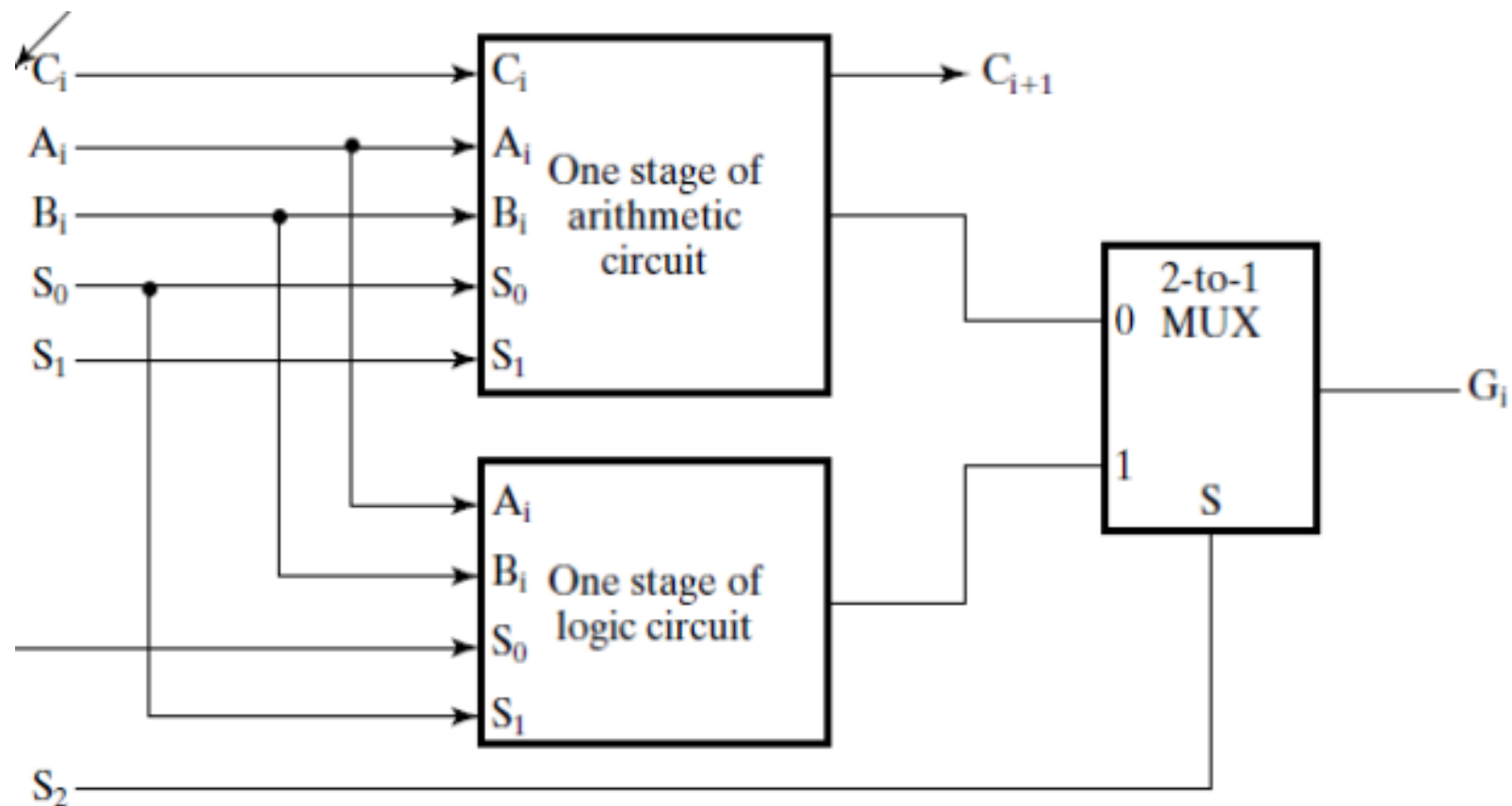


(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$G = A \wedge B$	AND
0	1	$G = A \vee B$	OR
1	0	$G = A \oplus B$	XOR
1	1	$G = \overline{A}$	NOT

(b) Function table

Arithmetic/Logic Unit: $C_{in} = S_2 \cdot C_i$



□ **FIGURE 7**
One Stage of ALU

□ TABLE 2
Function Table for ALU

Operation Select				Operation	Function
S_2	S_1	S_0	C_{in}		
0	0	0	0	$G = A$	Transfer A
0	0	0	1	$G = A + 1$	Increment A
0	0	1	0	$G = A + B$	Addition
0	0	1	1	$G = A + B + 1$	Add with carry input of 1
0	1	0	0	$G = A + \overline{B}$	A plus 1s complement of B
0	1	0	1	$G = A + \overline{B} + 1$	Subtraction
0	1	1	0	$G = A - 1$	Decrement A
0	1	1	1	$G = A$	Transfer A
1	X	0	0	$G = A \wedge B$	AND
1	X	0	1	$G = A \vee B$	OR
1	X	1	0	$G = A \oplus B$	XOR
1	X	1	1	$G = \overline{A}$	NOT (1s complement)

Assignment

- Design an ALU with 4 arithmetic & 2 logic operations.

Sheet

- Text Book Ch., 8